SRS 10 APRIL UPDATED

MoodMind.AI

AI-Based Mood and Mental Behavior Analysis from Chat Data

Prepared By:

- Noor Fatima BSCE23043
- Ayesha Abdul Samad BSCE23050
- Shaheer Faisal BSCE23012

Table of Contents

▼ Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions, Acronyms, and Abbreviations

1.4 References

1.5 Intended Audience

▼ Overall Description

2.1 Product Perspective

2.2 Product Functions

2.3 User Classes and Characteristics

2.4 Operating Environment

2.5 Design and Implementation Constraints

2.6 Assumptions and Dependencies

▼ Requirement Specification

3.1 External Interface Requirements

3.2 Functional Requirements (Scenario-Based)

3.3 Non-Functional Requirements

▼ System Constraints

▼ System Workflow

1. Introduction

1.1 Purpose

The purpose of MoodMind.AI is to analyze textual chat data using Natural Language Processing (NLP) techniques to detect emotions, sentiment, and behavioral patterns. The system provides insights into mood variations and identifies stress-related indicators.

1.2 Scope

The system will:

- Accept text input and chat files
- Perform sentiment analysis
- Detect emotions
- Analyze mood intensity
- Identify stress indicators
- Generate graphs
- Export CSV reports

This system is for educational purposes only and is not a medical diagnostic tool.

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
NLP	Natural Language Processing
AI	Artificial Intelligence
CSV	Comma Separated Values file format
Sentiment Analysis	Process of detecting emotional tone
Mood Intensity	Strength of emotional expression
Dataset	Collection of chat/text data

1.4 References

- Software Engineering Lecture Material (CE304T)
- TextBlob Documentation
- VADER Sentiment Library
- Python Official Documentation

1.5 Intended Audience

- Instructor
- Teaching Assistants
- Software Engineering Students
- NLP Researchers

2. Overall Description

2.1 Product Perspective

MoodMind.AI is a standalone AI-based system that processes chat data and generates emotional insights using NLP techniques.

2.2 Product Functions

- Text processing
- File upload handling
- Sentiment classification
- Emotion detection
- Mood analysis
- Stress detection
- Visualization
- CSV export

2.3 User Classes and Characteristics

User Type	Description
Basic User	Uploads text and views results
Researcher	Uses output for analysis
Admin (Future)	System management (not implemented)

2.4 Operating Environment

Component	Requirement
OS	Windows / Linux / MacOS
Language	Python 3.x
Libraries	TextBlob, VADER, Pandas, Matplotlib
Hardware	Minimum 4GB RAM, Intel i3+

2.5 Design and Implementation Constraints

- English language only
- No real-time processing
- Library-based NLP accuracy
- No external API usage

2.6 Assumptions and Dependencies

Assumption	Description
Valid Input	Users provide proper text/files
Installed Libraries	NLP tools are correctly installed
Offline System	No internet required
Data Quality	Output depends on input quality

3. Requirement Specification

3.1 External Interface Requirements

3.1.1 User Interface

- Text input box
- File upload option (.txt, .csv)
- Result dashboard
- Graph visualization panel
- CSV download button

3.1.2 Software Interfaces

- Python 3.x
- TextBlob / VADER
- Pandas
- Matplotlib

3.1.3 Hardware Interfaces

- Laptop/PC
- Minimum 4GB RAM

- Standard CPU

3.1.4 Communication Interfaces

- No network communication required
- Fully offline system

3.2 Functional Requirements

3.2.1 Use Cases

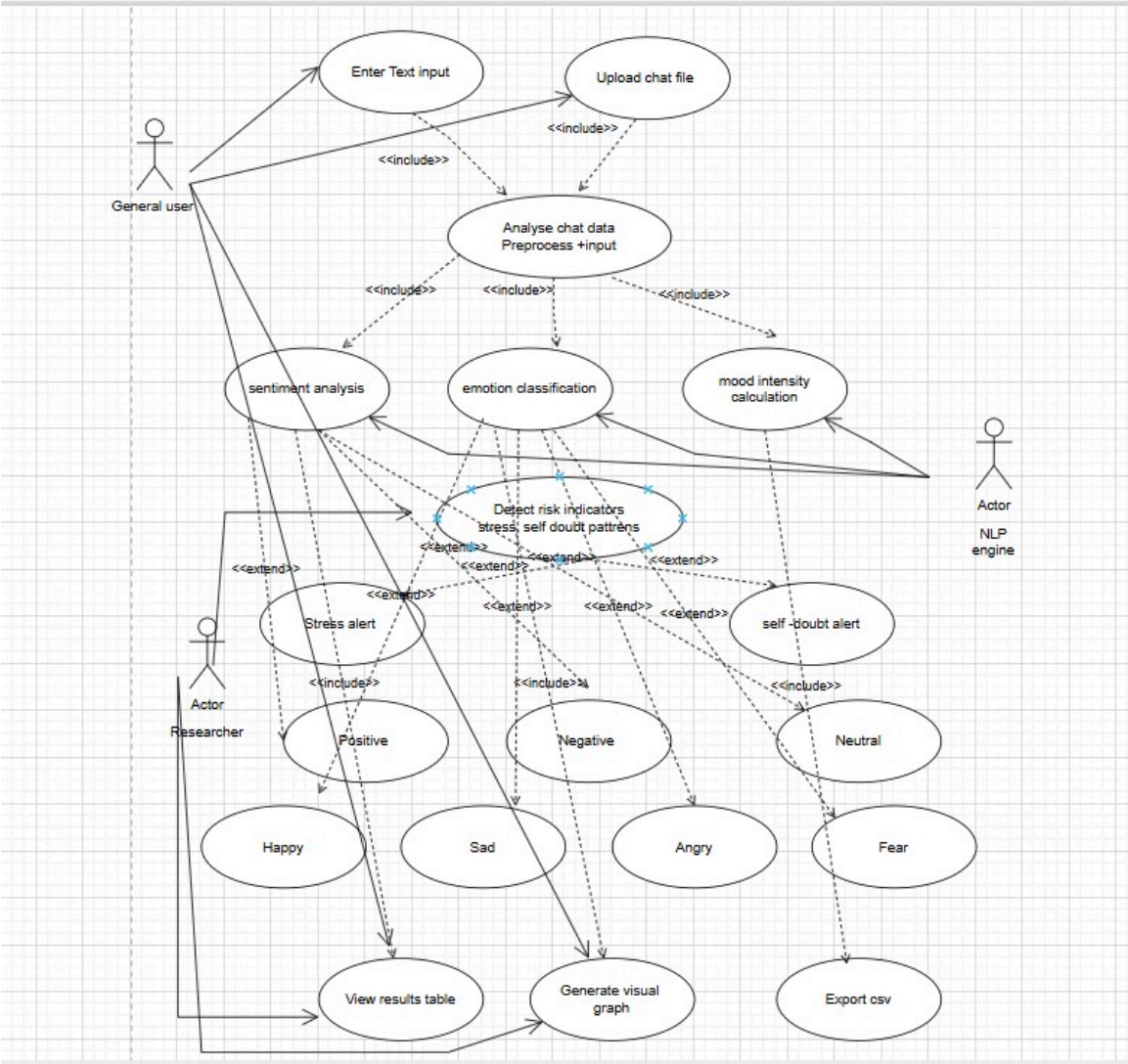

Figure 1: Use Case Diagram of MoodMind.AI System

Use Case Title	Description
Start System	User opens the system to begin analysis.
Input Text	User enters text for sentiment analysis.
Upload Chat File	User uploads chat dataset (.txt/.csv).
Sentiment Analysis	System identifies sentiment of text.
Emotion Detection	System detects emotional state from text.
Mood Intensity Analysis	System calculates strength of mood.
Stress Detection	System identifies stress patterns in text.
Visualization	System generates graphs and charts.
Export Results	User downloads results in CSV format.
Stop System	User closes the system.

3.2.2 Scenario-Based Functional Requirements

3.2.2.1 Scenario: Start System

Description:

User opens the system to begin analysis.

Functional Response:

The system initializes all modules, including the NLP engine, visualization tools, and user interface dashboard. It waits for user input.

3.2.2.2 Scenario: Input Text

Description:

User enters text into the system for analysis.

Functional Response:

The system receives text input, performs preprocessing (tokenization, cleaning), and forwards it to NLP models for sentiment and emotion detection.

3.2.2.3 Scenario: Upload Chat File

Description:

User uploads a chat file for processing.

Functional Response:

The system reads the file, extracts text messages, and processes each entry through NLP pipeline for analysis.

3.2.2.4 Scenario: Sentiment Analysis

Description:

System determines sentiment of input text.

Functional Response:

The system analyzes polarity of text and classifies it as Positive, Negative, or Neutral.

3.2.2.5 Scenario: Emotion Detection

Description:

System identifies emotion from text.

Functional Response:

The system processes text features and classifies emotions such as Happy, Sad, Angry, Fear, or Neutral.

3.2.2.6 Scenario: Mood Intensity Analysis

Description:

System measures strength of emotional expression.

Functional Response:

The system calculates sentiment score and determines intensity level (Low, Medium, High).

3.2.2.7 Scenario: Stress Detection

Description:

System detects stress-related patterns.

Functional Response:

The system identifies negative sentiment trends and stress-related keywords to detect emotional stress.

3.2.2.8 Scenario: Visualization

Description:

System generates graphical representation of results.

Functional Response:

The system uses Matplotlib to generate charts showing sentiment distribution and emotional breakdown.

3.2.2.9 Scenario: Export Results

Description:

User exports analyzed results.

Functional Response:

The system converts analysis output into CSV format and allows download for further use.

3.2.2.10 Scenario: Stop System

Description:

User closes the system.

Functional Response:

The system safely terminates all processes and clears temporary data.

3.3 Non-Functional Requirements

Category	Description
Performance	Efficient processing of small/medium datasets
Reliability	Consistent output and error handling
Security	No permanent storage, local processing
Maintainability	Modular design for easy updates
Reusability	Components reusable in other AI systems
Scalability	Extendable for larger datasets

4. System Constraints

Constraint	Description
Language	Only English supported
Processing	No real-time analysis
Accuracy	Depends on NLP models
Medical Use	Not a diagnostic tool
Performance	Slows with large datasets

5. System Workflow

Input → Preprocessing → NLP Processing → Sentiment Analysis → Emotion Detection → Mood Analysis → Visualization → Output (Graph + CSV)